

A Local Re-Decoding Algorithm for LDPC-Coded Modulation With CCDMs

Hanxiang Yu, Liyuan Song^{ID}, *Member, IEEE*, and Yi Wu^{ID}, *Member, IEEE*

Abstract—This letter proposes to break trapping sets of the low-density parity-check (LDPC) codes used in an LDPC-coded modulation system that employs constant composition distribution matchers (CCDMs). By re-decoding a list of candidate input message vectors, obtained in terms of the double extension of unsatisfied check nodes (UCNs) or the weight constraint of CCDM codewords, the LDPC decoder may get out of the trapped state. As a result, the shaping decoder of CCDM may acquire more accurate input messages from the re-decoding process of LDPC code. Simulation results show that the proposed local re-decoding (LRD) algorithm can achieve better performance with reasonable computational overhead compared with other algorithms for decoding LDPC codes in the LDPC-coded modulation system.

Index Terms—LDPC codes, CCDMs, error-floor, trapping set, re-decoding.

I. INTRODUCTION

HIGH-ORDER modulation using *probabilistic amplitude shaping* (PAS) combined with advanced channel coding schemes are critical enablers to achieve reliable communication of high data rates [1], [2]. *Constant composition distribution matchers* (CCDMs) and *low-density parity-check* (LDPC) codes have capacity-approaching capability. Therefore, they have been widely employed in systems with higher-order modulation [3]. However, most LDPC codes, especially for short code lengths, may exhibit the *error-floor* phenomenon under iterative decoding [4], [5], [6]. Due to the lack of error correction capability of the shaping decoder, the erroneous output word of LDPC decoder may cause severe error propagation in the shaping decoder. Thus, the error-floor of LDPC codes seriously limits the application of LDPC-coded modulation in systems which require very low error-rates, e.g., the *ultra-reliable low-latency communications* (URLLC) scenario whose the packet error probability is between 10^{-5} to 10^{-9} . A great deal of works have indicated that the error-floor of LDPC codes is mainly caused by a special undesirable structure, called as *trapping set* [5] or *near codeword* [4], in the Tanner graph of an LDPC code.

Received 26 November 2025; revised 30 December 2025; accepted 1 January 2026. Date of current version 9 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62201152; in part by the Fujian Province Special Fund Project for Promoting High-Quality Development of Marine and Fisheries Industries under Grant FJHYF-ZH-2023-03; and in part by the Fujian Provincial Direct Unit Special Project for Education and Research, Fujian Financial Referral [2023] No. 834. The associate editor coordinating the review of this article and approving it for publication was Y. C. Wu. (Hanxiang Yu and Liyuan Song equally contributed to this work.) (Corresponding author: Liyuan Song.)

The authors are with the Fujian Provincial Engineering Technology Research Center of Photoelectric Sensing Application, Key Laboratory of OptoElectronic Science and Technology for Medicine, Ministry of Education, Fujian Normal University, Fuzhou 350117, China (e-mail: qsx20230822@student.fjnu.edu.cn; songly@fjnu.edu.cn; wuyi@fjnu.edu.cn).

Digital Object Identifier 10.1109/LWC.2026.3651252

Recently, many researchers have dedicated themselves to seeking methods [7], [8], [9], [10], [11], [12], [13], [14] to lower the error-floors of LDPC codes. Various post-processing decoding techniques [10], [12], [13], [14] are introduced to break trapping sets based on the decoding results in case of the *belief-propagation* (BP) decoding failure without the detailed knowledge of the trapping sets. In [10], Kang et al. provided an iterative *backtracking decoding* (BTD) algorithm to break trapping sets while decoding by successively flipping bits connected to *unsatisfied check nodes* (UCNs), i.e., the CNs whose parity-check equations are not satisfied. In [13], a *belief-propagation-list erasure decoding* (BP-LED) algorithm was proposed to lower the error-floors of LDPC codes by combining erasure correction technology and list decoding via *Gaussian elimination* (GE) to correct the chosen unreliable bits. However, these methods may not able to efficiently identify the bit positions to perturb, and mainly consider the binary phase shift keying (BPSK) modulation.

This letter proposes a new post-processing technique, called as *local re-decoding* (LRD) algorithm, to break trapping sets of LDPC codes in LDPC-coded modulation systems with CCDMs. Specifically, to efficiently select perturbing bit positions, when the original LDPC decoding failed, we present a double extension scheme of the UCNs to select the adjacent *variable nodes* (VNs) with small reliability as the candidate VNs. If there are no UCNs, i.e., the original decoder may converge to a near codeword, we can also select the VNs associated with unreliable bits based on the *weight constraint* of CCDM codewords. Then, various combinations of candidate VNs are used to locally revise input messages of corresponding unreliable bits and generate a list of *candidate input message vectors* (CIMVs), i.e., each input vector of the LDPC decoder during post-processing. By re-decoding these CIMVs in turn, the LRD algorithm can help the LDPC decoder get out the trapped state and converge to a valid codeword, thereby improving the accuracy of input messages for the shaping decoder of CCDMs. Analysis results show that the LRD algorithm can lower the error-floors of LDPC codes with reasonable overhead for the LDPC-coded modulation system.

II. PRELIMINARIES

A. LDPC-Coded Modulation

Suppose that $\mathcal{C}$ is an LDPC code defined by the null space of an $m_c \times n$ parity-check matrix $\mathbf{H}$. Each column in $\mathbf{H}$ corresponds to one codeword bit and one VN in the Tanner graph of $\mathcal{C}$, and each row in $\mathbf{H}$ corresponds to one parity-check sum constraint and one *check node* (CN) in the Tanner graph. The Tanner graph of $\mathcal{C}$ can be denoted by $G = (V \cup C, E)$, where $V = \{V_0, V_1, \dots, V_{n-1}\}$ is the set of

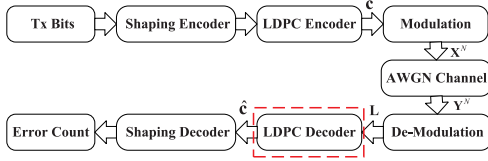


Fig. 1. Block diagram for the PAS model.

all VNs, $C = \{C_0, C_1, \dots, C_{m_c-1}\}$ is the set of all CNs, and E is the set of all edges connecting VNs and CNs.

Figure 1 shows a block diagram for the PAS model in the *additive white Gaussian noise* (AWGN) channel. In shaping encoder, k_s uniformly distributed data bits are mapped to n_s amplitudes of 2^m -Amplitude Shift Keying (ASK) modulation, which is generated from the CCDM based on the expected distribution. Each shaped amplitude is transformed into $m-1$ bits using Gray labeling to obtain a CCDM codeword of length $n_s(m-1)$. The number of non-zero bits in each CCDM codeword is a constant number w_s , which is called the weight constraint of CCDM codewords [3]. Suppose that $\mathbf{c} = (c_0, \dots, c_{n-1})$ is a codeword of $\mathcal{C}$, whose part of systematic bits correspond to all bits of several CCDM codewords. Then, each $m-1$ bits of these CCDM codewords are mapped to one amplitude of 2^m -ASK symbol using Gray labeling, and the corresponding sign bit comes from one bit of the non-CCDM codewords part of $\mathbf{c}$. Thus, $\mathbf{c}$ is mapped into a 2^m -ASK symbol sequence $\mathbf{X}^N$, where $N = n/m$. Obviously, the above processes can be directly extended to the *quadrature amplitude modulation* (QAM). The $\mathbf{X}^N$ is transmitted over the AWGN channel, and the output of channel is denoted by $\mathbf{Y}^N$.

The demodulator outputs the soft-decision value of each modulation symbol, i.e., the *log likelihood ratio* (LLR) value,

$$\text{which is calculated by } LLR_j = \ln \left(\frac{\sum_{x_j=0} p(y|x)p(x)}{\sum_{x_j=1} p(y|x)p(x)} \right), j =$$

$1, 2, \dots, m$, where $x \in \mathbf{X}^N$, $y \in \mathbf{Y}^N$, $p(y|x)$ is the conditional probability density function that depends on the specific channel, and $p(x)$ represents the prior probability of x . Here, $x_j = 0$ indicates that the j -th bit of x is 0. After bit-interleaving, the LLR values corresponding to all codeword bits of $\mathbf{c}$ can be obtained and are denoted by L_j for $0 \leq j < n$. The vector $\mathbf{L} = (L_0, L_1, \dots, L_{n-1})$ is the initial input message vector for the LDPC decoder. The absolute value of LLR of each bit, i.e., reliability measure, represents its reliability. Based on the Tanner graph of $\mathcal{C}$, the iterative decoding updates and passes reliability messages along the edges connecting VNs and CNs. Finally, the output hard-decision word $\hat{\mathbf{c}}$ of the LDPC decoder is input into the shaping decoder to recover the original data bits.

B. Trapping Sets

A trapping set [5] is a special structure in the Tanner graph $G = (V \cup C, E)$ which will lead to the error-floor of $\mathcal{C}$. An (a, b) trapping set is a set A of a VNs in G which induces a subgraph $G_t = (A \cup C_t, E_t)$ that contains b odd-degree CNs, where C_t is a set of all CNs adjacent to the VNs in A , and E_t is the set of edges. Let B be a set of all odd-degree CNs in C_t , where $B \subset C_t$ and $|B| = b$. For conventional iterative

decoding algorithms of LDPC codes, the trapping sets of smaller sizes are more harmful to error-floors [7], [10], [13].

If the failure of one LDPC decoder is caused by a trapping set, it may converge to an unchanged error state or oscillate among a set of error states during the decoding iterations. Error bits in a trapping set will cause all odd-degree CNs of this trapping set to be unsatisfied, i.e., their corresponding check-sums are not equal to zero, while other CNs connected to this trapping set will be satisfied and called as *satisfied CNs* (SCNs). Set the maximum iteration number as I_{max} . In the k -th decoding iteration, $1 \leq k \leq I_{max}$, if the current decoder state, i.e., the hard-decision word obtained in this iteration, is not a codeword of the code $\mathcal{C}$, there exists a set of the UCNs. After I_{max} iterations, if the LDPC decoder still can not converge to a valid codeword, i.e., $\hat{\mathbf{c}} \cdot \mathbf{H}^T \neq 0$, it is failed.

It is worth mentioning that the LDPC decoder may converge to a near codeword, i.e., $\hat{\mathbf{c}} \cdot \mathbf{H}^T = 0$ but $\hat{\mathbf{c}} \neq \mathbf{c}$, which is caused by a class of special trapping sets (the number of UCNs equals zero). Since some bits of the transmitted LDPC codeword $\mathbf{c}$ correspond to the encoded CCDM codewords, we can further check whether the word $\hat{\mathbf{c}}$ is correct or not based on the weight constraint of CCDM codewords. Specifically, if the weight of certain bits of $\hat{\mathbf{c}}$ associated with a CCDM codeword is not equal to w_s , it indicates that $\hat{\mathbf{c}} \neq \mathbf{c}$ and the decoder is failed.

III. LOCAL RE-DECODING ALGORITHM

As described in many excellent works [10], [13], it is possible to identify if the decoding failure was caused by the trapping set based on the UCNs. Furthermore, due to the weight constraint of CCDM codewords, the decoding failure caused by the near codeword would also be identified. Thus, this letter proposes the LRD algorithm to efficiently select candidate VNs which may belong to the small trapping sets or near codewords in terms of double extension of the UCNs or the unreliable bits of CCDM codewords. Then the original LLR values of unreliable bits are locally revised to obtain a list of CIMVs. By re-decoding CIMVs in turn, the LRD algorithm effectively breaks trapping sets with reasonable overhead.

A. Selection of Candidate VNs

Recall the LDPC code $\mathcal{C}$, CCDM codewords and small trapping sets defined in Section II. If the LDPC decoder fails, the smallest set of UCNs among all I_{max} iterations is denoted by $T_0 = \{C_{\iota_0}, C_{\iota_1}, \dots, C_{\iota_{|T_0|-1}}\}$, where $\iota_0, \iota_1, \dots, \iota_{|T_0|-1}$ are the indices of these UCNs in the Tanner graph G of $\mathcal{C}$. Let $\mathbf{R}$ be the output soft-decision message vector of length n of an iteration that obtains the smallest set T_0 .

If $0 < |T_0| \leq \tau$, τ is a predetermined threshold, it can be considered that the decoding failure was caused by a small trapping set. According to the set T_0 and $\mathbf{R}$, we can further identify the VNs in G of $\mathcal{C}$ that are likely to correspond to erroneous bits in a small trapping set, which are connected to a small number of UCNs and some SCNs. For $0 \leq i < |T_0|$, each UCN $C_{\iota_i} \in T_0$ can be doubly extended to obtain a *four-layer tree* Tr_i based on the connected relations of CNs and VNs in G . The root of the tree Tr_i is C_{ι_i} . We select candidate VNs based on the reliability measures of their corresponding bits in $\mathbf{R}$. However, we do not consider all adjacent VNs of

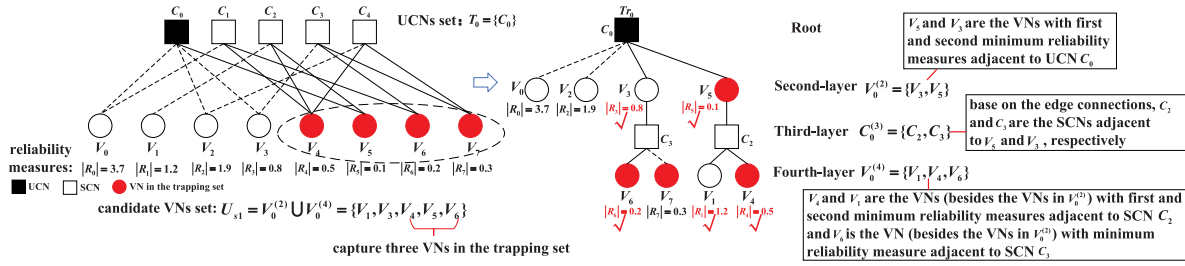


Fig. 2. Double extension of the UCNs in a (4,1) trapping set for an (8,3) LDPC code with maximum column weight of 3 and maximum row weight of 4.

C_{li} in order to narrow the field of VNs. Thus, the second-layer nodes of Tr_i are formed of the two VNs with the smallest reliability measures among all VNs connected to C_{li} , which are likely to be less reliable than the other adjacent VNs. These VNs comprise the set $V_i^{(2)}$. The set $C_i^{(3)}$ of all adjacent SCNs of each VN in $V_i^{(2)}$ forms the third-layer nodes of Tr_i .

To reduce complexity, for each CN in $C_i^{(3)}$ whose adjacent VN in $V_i^{(2)}$ with the minimum reliability measure, we only consider the two VNs with the smallest reliability measures among all adjacent VNs of this CN (besides the VNs in $V_i^{(2)}$). For each of the remaining CN in $C_i^{(3)}$, only the VN with the minimum reliability measure among all adjacent VNs of this CN (besides the VNs in $V_i^{(2)}$) is selected. Thus, each CN in $C_i^{(3)}$ is connected to one or two VNs in the fourth-layer nodes of Tr_i . These VNs in the fourth-layer comprise the set $V_i^{(4)}$. Last, we combine the VNs in the second and fourth layers of these $|T_0|$ trees to obtain a set $U_{s1} = \bigcup_{i=0}^{|T_0|-1} (V_i^{(2)} \cup V_i^{(4)})$. An example of the double extension of the UCNs in a (4,1) trapping set is given in Fig. 2. It shows that it can capture three VNs in the (4,1) trapping set.

If the final hard-decision word of LDPC decoder satisfies all parity-check equations, i.e., $|T_0| = 0$, but some bits associated with the CCDM codewords do not fulfill the weight constraint, it also declares a decoding failure. Then, we can also select another candidate VNs corresponding to unreliable bits of LDPC codeword based on the vector $\mathbf{R}$. Here, $\mathbf{R}$ is the final output soft-decision message vector of the LDPC decoder. By sorting the n -length vector $\mathbf{R}$ in ascending order of the reliability measures, the N_{max1} VNs associated with the bits of the first N_{max1} minimum reliability measures in $\mathbf{R}$ are selected, where N_{max1} is a predetermined value. These candidate VNs comprise the set U_{s2} . Then, we can further introduce additional operations to correct error bits in trapping sets based on the set U_s of all candidate VNs, where $U_s = U_{s1}$ for $0 < |T_0| \leq \tau$, otherwise $U_s = U_{s2}$ for $|T_0| = 0$.

B. LRD Algorithm

This section describes how to lower the error-floors of the LDPC codes based on the obtained set U_s . First, we would like to define some related notations used in the LRD algorithm. The maximum of all reliability measures in the initial input message vector $\mathbf{L}$ is denoted by $|L_{max}|$. Define the hard-decision bits of the vector $\mathbf{R}$ corresponding to all candidate VNs from U_s by a tuple $\mathbf{u}_s =$

$(u_{s,0}, u_{s,1}, \dots, u_{s,|U_s|-1})$, the LLRs of posterior probabilities of the bits in $\mathbf{u}_s$ (selected from $\mathbf{R}$) are represented by the vector $\mathbf{R}_s = (R_{\psi_0}, R_{\psi_1}, \dots, R_{\psi_{|U_s|-1}})$, whose elements have been rearranged in ascending order of their absolute values, i.e., $|R_{\psi_0}| \leq |R_{\psi_1}| \leq \dots \leq |R_{\psi_{|U_s|-1}}|$, where $\psi_0, \psi_1, \dots, \psi_{|U_s|-1}$ denote the indices of the VNs corresponding to $\mathbf{u}_s$ in the Tanner graph G . Let $\mathcal{A}_0 = \{\psi_0, \psi_1\}, \dots, \mathcal{A}_{g-1} = \{\psi_{|U_s|-2}, \psi_{|U_s|-1}\}$ be the sets that contain the pairwise indices, respectively, where $g = \binom{|U_s|}{2}$.

Since one hard-decision bit of smaller reliability measure is more possible to be erroneous, we consider the hard-decision bits in $\mathbf{u}_s$ as the unreliable bits and locally revise their initial reliability messages in the vector $\mathbf{L}$ based on the vector $\mathbf{R}_s$. First, we sequentially revise the reliability message of each unreliable bit in $\mathbf{L}$ to generate N_1 CIMVs, where $N_1 = \min\{|U_s|, N_{max1}\}$. Since some hard-decision bits with smaller reliability measures may also be correct, we further locally revise the reliability messages of these bits in $\mathbf{L}$. Due to a higher probability of falling into pseudo-codewords caused by overestimation of measurements, only the reliability messages associated with the first $N_2 = \min\{g, N_{max2}\}$ pairs of VNs in the set $\mathcal{A}_i$, $0 \leq i < N_2$, are sequentially modified to generate additional N_2 CIMVs, where N_{max2} is a predetermined value. The above $N_1 + N_2$ CIMVs are defined by $\mathbf{R}_c^{(i)} = (R_{c,0}^{(i)}, R_{c,1}^{(i)}, \dots, R_{c,n-1}^{(i)})$, where $0 \leq i < N_1 + N_2$.

Each CIMV is initialized with $\mathbf{L}$. For $0 \leq i < N_1$, the value $R_{c,\psi_i}^{(i)}$ in $\mathbf{R}_c^{(i)}$ is revised as $R_{c,\psi_i}^{(i)} = -1 * \text{sgn}(R_{\psi_i}) * |L_{max}| * \eta_1$, where η_1 is a scaling factor, and $\text{sgn}(x)$ is the sign of x . Then, for $N_1 \leq i < N_1 + N_2$, each value $R_{c,\alpha_j}^{(i)}$ in $\mathbf{R}_c^{(i)}$, $\alpha_j \in \mathcal{A}_{i-N_1}$, is revised as $R_{c,\alpha_j}^{(i)} = \text{sgn}(R_{\alpha_j}) * |L_{max}| * \eta_2$, where η_2 is a scaling factor. To avoid the overestimation of measurements and accelerate convergence, it generally sets $\eta_2 > \eta_1$. Last, these CIMVs are further re-decoded in order. A detailed description of the LRD algorithm is given in Algorithm 1.

C. Computational Complexity

We analyze the computational complexity of the LRD algorithm in case of the original LDPC decoding failure. In this letter, we employ the *min-sum algorithm* (MSA) as both the original decoding and re-decoding method for the LRD algorithm. Let P_{e1} indicate the probability that the original LDPC decoder is failed at a specific *signal-to-noise ratio* (SNR). P_{e2} indicates the probability that the first N_1 re-decoding fails. Consider the LDPC code $\mathcal{C}$ described in Section II. Let ρ and γ be the average row weight and

Algorithm 1: LRD Algorithm

-
- 1 **Input:** Initial input vector $\mathbf{L}$ of the original decoding and the vector $\mathbf{R}_s = (R_{\psi_0}, R_{\psi_1}, \dots, R_{\psi_{|U_s|-1}})$. The sets $\mathcal{A}_i$, $0 \leq i < N_2$, of pairwise indices.
 - 2 **for** $i = 0, \dots, N_1 - 1$ **do**
 - 3 $\mathbf{R}_c^{(i)} \leftarrow \mathbf{L}$ and $R_{c,\psi_i}^{(i)} \leftarrow -1 * \text{sgn}(R_{\psi_i}) \cdot |L_{\max}| \cdot \eta_1$;
 - 4 go to Step 1;
 - 5 **for** $i = N_1, \dots, N_1 + N_2 - 1$ **do**
 - 6 $\mathbf{R}_c^{(i)} \leftarrow \mathbf{L}$ and $R_{c,\alpha_j}^{(i)} \leftarrow \text{sgn}(R_{\alpha_j}) \cdot |L_{\max}| \cdot \eta_2$ for each $\alpha_j \in \mathcal{A}_{i-N_1}$;
 - 7 go to Step 1;
 - 8 **Step 1:** Perform re-decoding with the input vector $\mathbf{R}_c^{(i)}$. If a valid codeword satisfying CCDDM weight constraint is found, then exit and send the output codeword to the shaping decoder.
 - 9 **If** all re-decodings fail **then** exit and send the original hard-decision word to the shaping decoder.
-

column weight of the LDPC code $\mathcal{C}$, respectively. Let $I_{\max 1}$ and $I_{\max 2}$ be the maximum iteration number of the original decoding and each re-decoding, respectively. It is obvious that the complexity of the LRD algorithm is mainly determined by the predetermined values of $N_{\max 1}$ and $N_{\max 2}$ for re-decodings. Thus, we can neglect the complexity of double extension of the UCNs and generation of the list of CIMVs here.

Consider the worst-case that $|T_0| = \tau$, then $|U_s| \leq (3\gamma - 1)\tau$ and the maximum value of $N_{\max 1}$ is equal to $(3\gamma - 1)\tau$. The maximum value of $N_{\max 2}$ is equal to $\frac{1}{2}|U_s|^2 - \frac{1}{2}|U_s|$, i.e., $\frac{\tau^2}{2}(3\gamma - 1)^2 - \frac{\tau}{2}(3\gamma - 1)$. The post-processing of the LRD algorithm is invoked mainly in the error-floor region, where the actual probability P_{e1} is relatively low, for example, $P_{e1} < 10^{-4}$. Thus, in the high SNR regions, the actual average iteration number required for the re-decodings of MSA in the LRD algorithm is at most $P_{e1}(N_{\max 1} + P_{e2}N_{\max 2})I_{\max 2}$, which can be negligible when choosing the moderate values of τ . In the low SNR regions, the number of UCNs may be larger than the threshold τ , such that it will not start the re-decoding process. Thus, the additional computational complexity for the LRD algorithm is also acceptable in the low SNR regions.

IV. SIMULATION RESULTS

We compare the performance of the proposed LRD algorithm with other decoding algorithms of LDPC codes over the AWGN channel using probabilistically shaped 16-QAM with CCDDM. For comparison, we also use the *sum-product algorithm* (SPA), MSA, *BTD with first back-tracking* (1st-BTD) [10], *alternating direction method of multipliers* (ADMM) decoding [11], BP-LED algorithm [13], and *improved offset min-sum* (IOMS) algorithm [14], respectively, as the decoding methods for LDPC codes in the systems. The parameters of the ADMM and BP-LED algorithms are the same as those in [11] and [13], respectively. The

maximum iteration number $I_{\max 1}$ for the original decoding of the LRD, 1st-BTD, and BP-LED algorithms is set to 50. The maximum iteration number $I_{\max 2}$ of each re-decoding is set to 20 and 50 for the LRD and 1st-BTD algorithms, respectively. The weight constraint of CCDDM codewords is also used to detect near codewords during the post-processing of the 1st-BTD and BP-LED algorithms. Generally, the trapping sets that cause the failure decoding are typically small trapping sets [10], thus we set $\tau = 10$, $N_{\max 1} = 15$ and $N_{\max 2} = 35$ for the LRD algorithm in all simulations to reduce the number of the re-decodings. The threshold τ_{bt} for the number of UCNs, as defined in [10], is set to 10 for the 1st-BTD algorithm. The maximum iteration number of SPA/MSA/IOMS is set to $I_{\max 1} + I_{\max 2} * (N_{\max 1} + N_{\max 2}) = 1050$. The output of shaping decoder is used to evaluate the error performance, i.e., the *frame error rate* (FER). The “scaling” factors used in various algorithms are optimized by computer search under the metric of FER. Here, we set the scaling factor of the MSA to 0.815. Set $\eta_1 = 1.5$ and $\eta_2 = 1.75$ for the LRD algorithm.

For LRD and 1st-BTD, we also count the average number of re-decodings required in each post-processing, denoted by N_{avg} , the average number of additional iterations among all simulated T_1 codewords, denoted by ΔI_{avg} , and the average decoding runtime rt for each codeword implemented in MATLAB[®] 2024 at specific SNRs. Here, N_{avg} is calculated by dividing the total number of re-decodings among T_1 codewords by T_2 , where T_2 is the total number of post-processing.

Example 1: Consider the IEEE 802.16e WiMax standard (576, 288) LDPC code $\mathcal{C}_1$ with the code rate of 0.5, which exhibits high error-floors due to a large number of columns of weight 2 in their base matrix. Set $k_s = 6$, $n_s = 12$ and $\omega_s = 2$ for the CCDDM. The transmission rate is 1 bit/symbol. Figure 3 (a) shows that the error-floor of $\mathcal{C}_1$ is observed at the FER of 10^{-5} by using the SPA, MSA, IOMS and ADMM in the LDPC decoder, respectively. The proposed LRD algorithm further lowers the error-floor and outperforms the 1st-BTD/BP-LED algorithm for 0.27/0.55 dB at the FER of 10^{-6} . In Fig. 3 (b), the overall average iteration number of the LRD algorithm is essentially equivalent to that of the MSA/SPA starting from SNR of 4.8 dB. Table I shows that the LRD algorithm takes smaller values of N_{avg} , ΔI_{avg} and rt than the 1st-BTD at SNR of 5.8 dB. The number of GE operations per post-processing and decoding runtime of BP-LED are also given in Table I, which are larger than N_{avg} and rt of LRD, respectively. We further found that the dominant trapping set classes of the original MSA are (11, 2) and (13, 1) classes, and other non-negligible trapping set classes include (10, 2), (13, 2), and (16, 1) classes at SNR of 5.8 dB. Moreover, a (13, 0) class, i.e., the codewords with the smallest weight, is found that accounts for about 10% of the FER at SNR of 5.8 dB. While the LRD algorithm can break most of (11, 2), (13, 1), (10, 2) and (13, 0) trapping set classes. Last, the error-floor of our algorithm is estimated based on [6]. Figure 3 (a) shows that the estimation results are close to the simulation results.

Example 2: Consider a (272, 191) shorted *finite geometry* (FG)-LDPC code $\mathcal{C}_2$ with a code rate of 0.7, column weight of 4, and row weights of 13 and 14. Set $k_s = 4$, $n_s = 8$ and $\omega_s = 2$ for the CCDDM. The transmission rate is 1.8

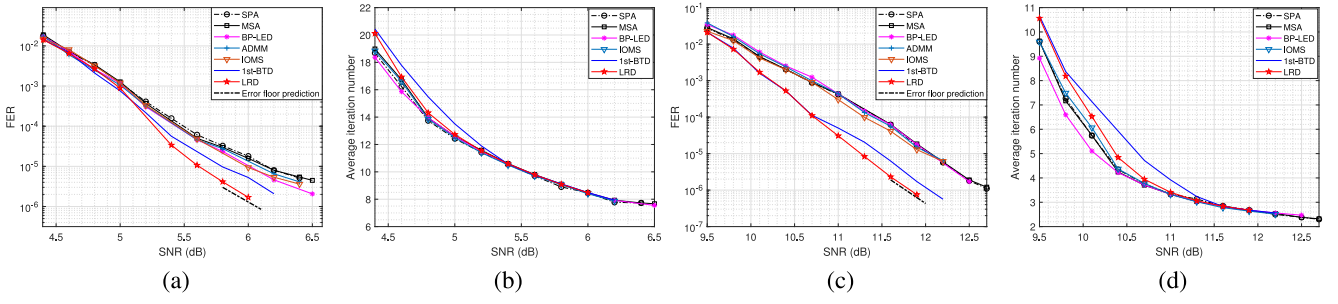


Fig. 3. (a) FERs of the (576, 288) LDPC code C_1 (b) Comparison of overall average iteration number for the (576, 288) LDPC code C_1 (c) FERs of the (272, 191) LDPC code C_2 (d) Comparison of overall average iteration number for the (272, 191) LDPC code C_2 .

TABLE I
NUMBER OF RE-DECODINGS/GE PER POST-PROCESSING AND AVERAGE RUNTIME OF THE CORRESPONDING DECODERS IN EXAMPLES 1-2

Code	Decoder	T_1	T_2	N_{avg}	I_2	ΔI_{avg}	G_1	rt
C_1 (5.8 dB)	BP-LED	10^6	28	/	/	/	15	22.7
	1st BTD	10^6	14	8.1	50	0.006	/	18.2
	LRD	10^6	23	6.3	20	0.003	/	16.7
C_2 (11.9 dB)	BP-LED	10^7	144	/	/	/	15	2.74
	1st BTD	10^7	134	9.9	50	0.007	/	2.31
	LRD	10^7	139	8.4	20	0.002	/	2.22

T_1 : Number of simulated codewords;

T_2 : Total number of post-processing among T_1 codewords;

N_{avg} : Average number of re-decodings per post-processing;

I_2 : Maximum iteration number per re-decoding; $\Delta I_{avg} = (T_2/T_1) * N_{avg} * I_2$;

G_1 : Number of GE per post-processing; rt (ms): Average runtime per codeword.

bits/symbol. In Fig. 3 (c), when using the MSA/SPA/BP-LED, it is observed the error-floor at the FER of 10^{-3} , respectively. However, the LRD algorithm further lowers the error-floor and outperforms the 1st-BTD for about 0.3 dB at the FER of 10^{-6} . Figure 3 (d) shows that the LRD algorithm has a slightly higher average iteration number compared to the MSA/SPA in the waterfall region. It is because there are some error bits not caused by the trapping sets which may also be invoked in the re-decodings of LRD for C_2 . Thus, the LRD algorithm further improves the waterfall performance. With the increase of SNR, e.g., starting from 11.3 dB, the average iteration number for all algorithms becomes essentially the same. Table I shows that the LRD algorithm takes less $N_{avg}/\Delta I_{avg}/rt$ than 1st-BTD at SNR of 11.9 dB. Thus, the extra overhead of the LRD algorithm is acceptable in the error-floor region. At SNR of 11.6 dB, the dominant trapping set classes in the original MSA are found to be three (5, 4) classes, and other non-negligible trapping set classes include (6, 6), (7, 2), (7, 4), and (6, 4) classes. The LRD algorithm can break most of (5, 4) and (6, 6) trapping set classes. Figure 3 (c) shows that the error-floor prediction of our algorithm matches with the simulation results.

V. CONCLUSION

This letter has devised a re-decoding algorithm to locally revise the reliability messages of bits in harmful trapping sets based on an expanded subgraph of UCNs or unreliable bits of the CCDM codewords. By restarting the decoding for a list of CIMVs, the proposed algorithm can correct reminding errors and lower the error-floor of LDPC codes with moderate overhead in the LDPC-coded modulation system.

In future studies, it will be interesting to further exploit the results from different decoder iterations to adaptively adjust the parameters and input vectors for the re-decodings, which may further improve the error performance of our algorithm. Moreover, it is interesting to further investigate the asymptotic performance analysis on our algorithm by employing other analytical tools, e.g., EXIT charts.

REFERENCES

- [1] G. Böcherer, F. Steiner, and P. Schulte, "Bandwidth efficient and rate-matched low-density parity-check coded modulation," *IEEE Trans. Commun.*, vol. 63, no. 12, pp. 4651–4665, Dec. 2015.
- [2] T. Prinz et al., "Polar coded probabilistic amplitude shaping for short packets," in *Proc. Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, 2017, pp. 1–5.
- [3] Y. Luo and Q. Huang, "Joint message passing decoding for LDPC codes with CCDMs," *Opt. Lett.*, vol. 48, no. 19, pp. 4933–4936, Sep. 2023.
- [4] D. J. MacKay and M. S. Postol, "Weaknesses of Margulis and Ramanujan-Margulis low-density parity-check codes," *Electron. Notes Theor. Comput. Sci.*, vol. 74, pp. 97–104, Oct. 2003.
- [5] T. Richardson, "Error floors of LDPC codes," in *Proc. 41st Annu. Allerton Conf. Commun., Control Comp.*, vol. 41, Oct. 2003, pp. 1426–1435.
- [6] P. Neshastegaran, A. H. Banihashemi, and R. H. Gohary, "Error floor estimation of LDPC coded modulation systems using importance sampling," *IEEE Trans. Commun.*, vol. 69, no. 5, pp. 2784–2799, Feb. 2021.
- [7] R. Asvadi, A. H. Banihashemi, and M. Ahmadian-Attari, "Lowering the error floor of LDPC codes using cyclic liftings," *IEEE Trans. Inf. Theory*, vol. 57, no. 4, pp. 2213–2224, Jul. 2011.
- [8] E. Ben Yacoub, "Trapping and absorbing set enumerators for irregular generalized low-density parity-check code ensembles," *IEEE Trans. Inf. Theory*, vol. 69, no. 6, pp. 3637–3662, Jun. 2023.
- [9] L. Zou, H. Yan, J. Dong, Y. Li, P. Chen, and F. C. Lau, "On construction of low-density parity-check codes for ultra-reliable and low-latency communications," *IEEE Trans. Commun.*, vol. 72, no. 9, pp. 5290–5301, Apr. 2024.
- [10] J. Kang, Q. Huang, S. Lin, and K. Abdel-Ghaffar, "An iterative decoding algorithm with backtracking to lower the error-floors of LDPC codes," *IEEE Trans. Commun.*, vol. 59, no. 1, pp. 64–73, Jan. 2010.
- [11] H. Wei, X. Jiao, and J. Mu, "Reduced-complexity linear programming decoding based on ADMM for LDPC codes," *IEEE Commun. Lett.*, vol. 19, no. 6, pp. 909–912, Mar. 2015.
- [12] S. Scholl, P. Schläfer, and N. Wehn, "Saturated min-sum decoding: An 'afterburner' for LDPC decoder hardware," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2016, pp. 1219–1224.
- [13] I. E. Bocharova, B. D. Kudryashov, V. Skachek, and Y. Yakimenko, "BP-LED decoding algorithm for LDPC codes over AWGN channels," *IEEE Trans. Inf. Theory*, vol. 65, no. 3, pp. 1677–1693, Aug. 2018.
- [14] B. N. Tran-Thi, T. T. Nguyen-Ly, H. N. Hong, and T. Hoang, "An improved offset min-sum LDPC decoding algorithm for 5G new radio," in *Proc. Int. Symp. Electr. Electron. Eng. (ISEE)*, 2021, pp. 106–109.